Guía de contenido - Semana 2: Estructuras de datos nativas en Python y control de flujo

Contenidos

- Tipos de datos: numéricos, booleanos, cadenas de texto (strings).
- Variables y operadores (aritméticos, relacionales, lógicos y de asignación).
- Estructuras de control: if-else, elif, bucles for y while.
- Estructuras de datos nativas: listas, tuplas, conjuntos y diccionarios.
- Diferencias básicas entre cada estructura y sus métodos más usados.

1) Tipos de datos

Constituyen la naturaleza fundamental del dato, es decir, qué tipo de valor estamos manipulando: número, texto, lista, etc. En Python, todo es un objeto, y cada objeto pertenece a una clase que define su tipo. Existen varios tipos de datos fundamentales que se usan para representar la información. A continuación se presenta un resumen de los principales, con ejemplos sencillos para que puedas comprender su utilidad y diferencia:

1a) Tipos Numéricos

In [1]:
```
# Enteros (int)

a = 10
b = -5
print(a + b)

# Flotantes (float)

x = 3.14
y = 2.0
print(x * y)

#  Complejos (complex)

c = 2 + 3j
```

```
d = 1 - 1j
resultado = c * d
print(resultado)
```

```
5
6.28
(5+1j)
```

Como averiguo el tipo de dato de un objeto?

Para averiguar el tipo de datos de un objeto utilizamos el comando **type()**. Lo vamos a utilizar con cualquier objeto de Python. Es muy útil para depuración, conversion, validación o simplemente para entender el comportamiento de ciertas operaciones.

In [ ]:
```

```

```
Tu edad ingresada es: 56.3. El tipo de dato es:  <class 'str'>
```

Ejercicio 1.1: Manejando datos - Uso de **type()**

- Crea dos variables: *d* y *e* conteniendo los valores 5 y 8.3 respectivamente.
- Realiza la division *a/b*. Asigna el resultado a una nueva variable llamada *resultado*.
- Verifica el tipo de dato de *resultado* utilizando **type()**.
- Imprime el resultado

In [ ]:
```

```

Ejercicio 1.2: Manejando datos - Uso de **type()**

- Crea dos variables: *d* y *e* conteniendo los valores 5 y 2 + 3j respectivamente.
- Realiza la suma *d* + *e*. Asigna el resultado a una nueva variable llamada *resultado*.
- Verifica el tipo de dato de *d*, *e* y *resultado*, utilizando **type()**.
- Imprime los tipos de todas variables. Que observas?

In [ ]:
```

```

1b) Tipos Booleanos (bool)

In [2]:
```
# Booleanos (bool)

es_mayor = 10 > 5
print(es_mayor)
```

```
True
```

In [3]:
```
# Comparaciones

edad_Juan = 18
tu_edad =   55  # <- Completa con tu edad.
```

```
resultado = tu_edad >= edad_Juan
print(f'El resultado es: {resultado}')  # Imprime por pantalla el resultado (depend
```

```
El resultado es: True
```

1c) Tipo cadenas de Texto (str)

Las cadenas son secuencias de caracteres y se pueden definir entre comillas simples o dobles.

In [4]:
```
mensaje = "Hola, mundo"
otro_mensaje = 'Python es uno de los lenguajes mas utilizados en Ciencia de Datos'
print(mensaje)
print(otro_mensaje)
```

```
Hola, mundo
Python es uno de los lenguajes mas utilizados en Ciencia de Datos
```

Las cadenas admiten operaciones como concatenación y repetición:

In [5]:
```
saludo = "Hola"
nombre = "Ana"
mensaje_completo = saludo + " " + nombre
print(mensaje_completo)
print('')   # <- Imprime una linea en blanco
print("!" * 3)
```

```
Hola Ana

!!!
```

- Veamos el siguiente ejemplo de validacion de datos:

In [6]:
```
# Averiguo el tipo de dato ingresado (validacion)

a = "35"
print(f'Tu edad ingresada es: {a}. El tipo de dato es: ', type(a))
# Nota: valor simulado reemplazó una entrada con input()
```

```
Tu edad ingresada es: 35. El tipo de dato es:  <class 'str'>
```

- El ejemplo anterior es para tener en cuenta que todo lo que se ingresa via **input** se almacena con formato **string**. Para utilzar estos valores (posiblemente operar con ellos) es necesario convertirlos.

¿Qué es la conversión de datos en Python?

Existen veces que necesitamos **convertir** los tipos de datos. Un ejemplo practico ocurre en la importacion de archivos (el caso tipico es una planilla de cálculo), donde a veces las columnas numericas se importan al ambiente de Python con formato texto. Otro ejemplo se

da cuando tenemos que ingresar datos numericos por pantalla. Para ello se utilizan los siguientes constructores:

- **int()** : construye un número al tipo entero a partir de un numero entero dado en formato texto.
- **float()** : construye un número al tipo float dada su expansion decimal en formato texto.
- **str()**: convierte a texto otros formatos como int, float, etc.

In [7]:
```
# int()

x = int(1.7)
y = int('8')
z = int("3")

print(x)
print(y)
print(z)
```

```
1
8
3
```

In [8]:
```
# float()

x = float(1.3)
y = float('8')
z = float("3")

print(x)
print(y)
print(z)
```

```
1.3
8.0
3.0
```

In [9]:
```
# str()

x = str(1.3)
y = str('8')
z = str(False)

print(x)
print(y)
print(z)
```

```
1.3
8
False
```

Ejercicio 1.3

Dada la cadena: '2.8', quiero obtener como resultado el entero 2. ¿Qué falla en la linea de codigo? :

int('2.8')

¿Cómo lo solucionarías?

In [ ]:

Ejercicio 1.4: Comparaciones logicas y conversion de tipos de datos (**casteo**). Uso de **int()**, **float()**

Escribe un script que solicite el ingreso de dos numeros por pantalla, que luego realice las comparaciones entre ellos utilizando **>**, **<** o **==**, y que finalmente imprima por pantalla estas comparaciones. **Ayuda:** Recuerda convertir los numeros ingresados. Otro detalle importante a tener en cuenta es que al comparar variables booleanas entre si, para verificaf igualdad, no se utiliza **=**, sino **==**.

In [ ]:

1d) Tipos secuencias

Dentro de este grupo encontraremos esencialemetne tres tipos: Listas (**list**), t-uplas (**tuples**) y rango (**range**).

- Las **listas** son colecciones ordenadas y mutables que pueden contener elementos de diferentes tipos.
- Las **tuplas** son similares a las listas pero son inmutables, lo que significa que una vez creadas, no se pueden modificar.
- **range** es un tipo de dato especial que representa una secuencia de números enteros generada bajo demanda.

Veremos pronto varios ejemplos de aplicacion de estos tipos de datos a las estructuras de control de flujo de datos.

1d.1) Listas

In [10]:
```python
# Listas

numeros = [1, 2, 3, 4, 5]
mezcla = [10, "Python", 3.14, True]
print(numeros)
print(mezcla)

# Ejemplo de manipulación:
numeros.append(6)
print(numeros)  # Salida: [1, 2, 3, 4, 5, 6]
```

```
[1, 2, 3, 4, 5]
[10, 'Python', 3.14, True]
[1, 2, 3, 4, 5, 6]
```

Ejercicio 1.5: Listas

- Crea la lista **familia = ["Juan", 1.73, "Luis", 1.68, "Madre", 1.71, "Padre", 1.89]**
- Localiza el elemento en la posición 4 de la lista.
- Localiza el elemento en la última posición de la lista. Investiga una forma alternativa de hallarlo.

In [11]:
```
familia = ["Juan", 1.73, "Luis", 1.68, "Madre", 1.71, "Padre", 1.89]
```

Ejercicio 1.6: Manejo de listas. Uso de la técnica de **slicing**



- Considera la lista del ejercicio anterior. Que obtenemos al escribir: **familia[2:5]**?
- Y con **familia[:3]**?
- Y cuando escribimos **familia[4:]**?

In [ ]:
```

```

Ejercicio 1.7: Manejo de listas. Cambiar, agregar y remover elementos de una lista (Investigar)

- Cómo cambio el sexto elemento de la lista (1.71) por 1.83?
- Cómo cambio los elementos que están en la tercera y cuarta posición de la lista por ['Rubén', 1.76]?
- Cómo agrego al final de la lista los elementos ["Alberto", 1.93]?
- Usa el comando **del** para borrar el quinto elemento de la lista?
- Cómo borrarías los tres ultimos elementos de la lista usando solo una linea de codigo?

In [12]:
```
familia = ["Juan", 1.73, "Luis", 1.68, "Madre", 1.71, "Padre", 1.89]
familia[5] = 1.83
print(familia)
familia[2:4] = ['Rubén', 1.76]
print(familia)
familia = familia + ["Alberto", 1.93]
print(familia)
del(familia[4])
print(familia)
```

```
del(familia[-3:])
print(familia)
```

```
['Juan', 1.73, 'Luis', 1.68, 'Madre', 1.83, 'Padre', 1.89]
['Juan', 1.73, 'Rubén', 1.76, 'Madre', 1.83, 'Padre', 1.89]
['Juan', 1.73, 'Rubén', 1.76, 'Madre', 1.83, 'Padre', 1.89, 'Alberto', 1.93]
['Juan', 1.73, 'Rubén', 1.76, 1.83, 'Padre', 1.89, 'Alberto', 1.93]
['Juan', 1.73, 'Rubén', 1.76, 1.83, 'Padre']
```

Ejercicio 1.7: Manejo de listas. Lee el siguiente codigo y explica lo ocurrido (Investigar). Como se evita esto?

In [13]:
```
nombres = ['Luis', 'Pedro', 'Juan']
print(nombres)                        #<- Imprime lista original
supuesta_copia_nombres = nombres      #<- Almacena el contenido de la lista original
supuesta_copia_nombres[0] = 'Carlos' #<- Modifica el primer nombre en 'supuesta_cop
print(supuesta_copia_nombres)         #<- Imprime lista 'supuesta_copia_nombres'
print(nombres)                        #<- Por que se modifica la lista 'nombres'????
```

```
['Luis', 'Pedro', 'Juan']
['Carlos', 'Pedro', 'Juan']
['Carlos', 'Pedro', 'Juan']
```

- **Respuesta:**

- para concluir con las secuencias, mencionaremos brevemente los últimos dos tipos:

1d.2) t-uplas

In [14]:
```
# t-uplas

coordenadas = (10.0, 20.0)
print(coordenadas)
# Si intentamos cambiar un elemento, Python lanzará un error:
#coordenadas[0] = 5.0  # Esto daría error, porque la t-uplas son inmutables
```

```
(10.0, 20.0)
```

1d.3) range

In [15]:
```
# range

# Ejemplo de uso en construccion de listas.
lista = list(range(1,5))  # Notemos que al construir la lista, NO incluye el ultimo
print(lista)
```

```
[1, 2, 3, 4]
```

1d) Tipo mapeo: Diccionarios (dict)

Los diccionarios son colecciones de pares clave-valor. Son mutables y permiten acceso rápido a los datos mediante sus claves.

In [16]:
```python
# Diccionario

estudiante = {
    "nombre": "Juan",
    "edad": 21,
    "carrera": "Ingeniería"
}
print(estudiante)
print(estudiante["nombre"])

# Ejemplo de actualización:
estudiante["edad"] = 22
print(estudiante["edad"])
```

```
{'nombre': 'Juan', 'edad': 21, 'carrera': 'Ingeniería'}
Juan
22
```

Ejercicio 1.6: Uso de diccionarios

Solicite al usuario los siguientes datos correspondientes a una sustancia química:

- Nombre de la sustancia
- Masa molar (en g/mol)
- Estado físico a temperatura ambiente (sólido, líquido o gaseoso)
- Almacene esta información en un diccionario cuyas claves sean: "nombre", "masa_molar" y "estado".

Imprima en pantalla el diccionario completo mostrando la información ingresada por el usuario.-

In [ ]:
```python

```

- Ejemplos: Mas acerca del uso de **type()**

In [17]:
```python
# Conversion de valores ingresados y posterior verificacion.

masa_molar = float("35")
print(f"La masa molar es: {masa_molar} y el tipo de dato:", type(int(masa_molar)))

# Nota: valor simulado reemplazó una entrada con input()
```

```
La masa molar es: 35.0 y el tipo de dato: <class 'int'>
```

In [18]:
```python
# Tipos numéricos
print(type(5))          # <class 'int'>
print(type(3.14))       # <class 'float'>
print(type(True))       # <class 'bool'>
```

```
# Cadenas de texto
print(type("Hola"))      # <class 'str'>

# Estructuras de datos
print(type([1, 2, 3]))   # <class 'list'>
print(type((1, 2, 3)))   # <class 'tuple'>
print(type({"a": 1}))    # <class 'dict'>
```

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
```

Poder saber el tipo de dato es esencial, sobre todo luego de la importacion de datos desde, por ejemplo, planillas de calculo o archivos csv. Muchas veces es necesario reformatear los datos antes de seguir trabajando con ellos.

1e) Otros tipos de datos

Existen datos de otras clases, como **bytes** (tipo binario), **set** (tipo conjunto), que son menos utilizados en muchas aplicaciones, se exploraran en caso de ser necesarios utilizando el **help** de Python.

2) Variables y operadores (aritméticos, relacionales, lógicos y de asignación)

En la clase anterior analizamos los distintos tipos de operadores: aritméticos, relacionales y booleanos. Recordamos que los operadores aritméticos —conocidos desde la escuela— generan resultados numéricos al realizar operaciones. En cambio, los operadores relacionales y booleanos devuelven valores de tipo booleano, es decir, True o False, según se cumpla o no una determinada condición. Los ejemplos correspondientes a los operadores aritméticos ya fueron presentados en el cuadro de la clase anterior. En esta ocasión, centraremos nuestra atención en ejemplos prácticos del uso de operadores relacionales, booleanos y de asignación.

2a) Operadores booleanos y relacionales

Respecto de las operaciones booleanas en Python, tenemos el siguiente cuadro para tener como referencia:

Operación	Ejemplo	Resultado	Explicación
`and`	`True and True`	`True`	Solo es `True` si **ambos operandos** son `True`
	`True and False`	`False`	
	`False and True`	`False`	
	`False and False`	`False`	
`or`	`True or True`	`True`	Es `True` si **al menos uno** de los operandos es `True`
	`True or False`	`True`	
	`False or True`	`True`	
	`False or False`	`False`	
`not`	`not True`	`False`	Invierte el valor lógico
	`not False`	`True`	

In [19]:
```python
# Ejemplos de operadores relacionales

# Asignamos valores a las variables
a = 15
b = 3.5

# Comparaciones y resultados booleanos
print("¿a es igual a b?: ", (a == b))
print("¿a es distinto de b?: ",a != b)
print("¿a es mayor que b?: ",a > b)
print("¿a es menor que b?: ",a < b)
print("¿a es mayor o igual que b?: ",a >= b)
print("¿a es menor o igual que b?:",a <= b)
```

```
¿a es igual a b?:  False
¿a es distinto de b?:  True
¿a es mayor que b?:  True
¿a es menor que b?:  False
¿a es mayor o igual que b?:  True
¿a es menor o igual que b?: False
```

In [20]:
```python
# Ejemplos de uso de operadores relacionales y booleanos

# Asignamos valores a las variables
a = 5.3
b = 15
c = 9.78

# Comparaciones y combinaciones de condiciones
print("¿a es igual a b y c es mayor que a?: ", a == b and c > a) # No hacen falta l
print("¿a es menor que b y c es menor que b?: ", (a < b and c < b))
print("¿a es menor que b o c es mayor que b?: ", (a < b or c > b))
print("¿a es distinto de c y c es mayor o igual que b?: ", (a != c and c >= b))
print("¿c está entre a y b?: ", (a < c and c < b))  # c debe estar entre a y b
```

```
¿a es igual a b y c es mayor que a?:  False
¿a es menor que b y c es menor que b?:  True
¿a es menor que b o c es mayor que b?:  True
¿a es distinto de c y c es mayor o igual que b?:  False
¿c está entre a y b?:  True
```

Por ejemplo, para la primera comparacion del cuadro anterior, tenemos que **a = b** -> Falso, **c > a** -> Verdadero. Al consultar la tabla previa vemos que **False and True = False**, que es el resultado que arroja Python. El mismo analisis se debe hacer con cualquiera de las otras comparaciones. La utilidad de los operadores booleanos se verá sobre todo cuando generamos **filtros** o en la **agrupación** sobre un conjunto de datos para calcular métricas de interés posteriormente.

Ejercicio 2.1: Uso de operadores booleanos y relacionales

Una persona debe poseer ciertos requisitos minimos para ingresar al cuerpo de Policía de Seguridad Aeroportuaria (PSA). En caso de cumplirlos, posteriormente se incorporan a un proceso de selección posterior. Las reglas de filtrado previo de aspirantes son: Tener entre 18 y 25 años Altura mayor a 1.60 metros Peso menor de 75 kg Pais de origen Argentina o por opción (supongamos que esta condición no es excluyente). Escribe un script que compare las cuatro condiciones con los operadores apropiados, para ver si tú, con tus datos personales pasarías el filtro previo.

In [ ]:
```
```

2b) Operadores de asignación

Es muy frecuente que en el contrexto de un código sea necesario asignar un nuevo valor a una variable que tenía contenido. Por ejemplo, supongamos que tenemos **x = 4** y que luego incrementamos en 2 su valor. La nueva asignación es **x = x + 2**. El contenido actualizado de la variable será 6. Veamos el ejemplo:

In [21]:
```
x = 4
x = x + 2
print(x)
```

```
6
```

- Notemos que el signo **=** no significa lo mismo que en Matemática (no hay ningún valor de x que cumpla la ecuación x = x + 4). En programación el signo igual significa **asignación**.

Python tiene unos operadores **especiales** que implican asignación. Veamos un ejemplo:

In [22]:
```
x = 4
x += 2  # en lugar de x = x + 2
print(x)
```

```
# Otros ejemplos

y = 7
y *= 2  # en lugar de  y = 2 * y
print(y)

z = 5
z **= 2 # en lugar de z = z**2
print(z)
```

```
6
14
25
```

La siguiente es una lista de operadores de asignación comunes:

Operador	Significado	Ejemplo	Equivalente a...
`=`	Asignación simple	`x = 5`	Asigna el valor 5 a `x`
`+=`	Suma y asigna	`x += 3`	`x = x + 3`
`-=`	Resta y asigna	`x -= 2`	`x = x - 2`
`*=`	Multiplica y asigna	`x *= 4`	`x = x * 4`
`/=`	Divide y asigna (float)	`x /= 2`	`x = x / 2`
`//=`	División entera y asigna	`x //= 2`	`x = x // 2`
`%=`	Módulo y asigna	`x %= 3`	`x = x % 3`
`**=`	Potencia y asigna	`x **= 2`	`x = x ** 2`
`&=`	AND bit a bit y asigna	`x &= 1`	`x = x & 1`
`` ` ``	=`	OR bit a bit y asigna	`x
`^=`	XOR bit a bit y asigna	`x ^= 3`	`x = x ^ 3`
`>>=`	Desplazamiento a la derecha	`x >>= 1`	`x = x >> 1`
`<<=`	Desplazamiento a la izquierda	`x <<= 2`	`x = x << 2`

Ejercicio 2.2: Uso de operadores de asignacion

El precio de un artículo es $1500. Se desea hacer una actualización del precio del producto con el 5%. Cuál es el nuevo precio del mismo? Te mostramos una manera de hacerlo:

In [23]:
```
x = 1500
incremento = x*0.05
```

```
precio_actualizado = x + incremento
precio_actualizado
```

Out[23]: 1575.0

In [ ]:

Realiza los mismo, pero utilizando el operador de asignación *=.

In [24]:
```
x = 1500

# Completa el codigo:
```

3) Estructuras de control: if-else, elif, bucles for y while.

En Python, las **estructuras de control de flujo** permiten dirigir el curso de ejecución del programa, decidiendo qué instrucciones se ejecutan y cuándo.